"If I have seen further it is by standing on the shoulders of giants."
-Isaac Newton

"Words are a pretext. It is the inner bond that draws one person to another, not words."
— Jalaluddin Rumi

Chapter 3

Introduction To C++ Language

Generic Table Printing

Summary

Like human beings, computers have their own languages that we use to interact and operate. These languages are also called programming languages. C++ is one of the most famous and widely used programming languages. Just like human languages have grammar, computer languages have syntax[1] and logical rules to link the steps of instruction to form a computer program.

The hardware of the computer only understands the language of the 0's and 1's and this language is called **binary language**. This language is difficult for humans to understand, so computer scientists have made programs that convert an easier human-friendly language to binary language. Such a program is known as a **compiler**. Binary language is also called a "low-level" language, in the sense that it is close to the electronics of a computer. The programming languages that are easy for humans to understand are called "high-level" languages, and C++ is one such language.

Let us now dive into learning the basics of C++. *Always remember that the language constructs like syntactic(grammatical), morphological (keywords), and semantic rules will only teach you the tool to express your computational thought. Learning those computational ideas will be the most important journey you will experience throughout this course*.

Lesson # 1 Writing Our First Computer Program

Let us write our first C++ program which should write (we also use the word "print" for this) a message on our Console Window(black window). This will be the basic means of interaction between the end-user(the client who will be using our software) and our code. You may think of it as a blackboard where the running program will display some output. Also, the user can give some input to the running program by typing whatever the program needs using the keyboard. This, taking input from the console, is called **prompting** the user, after taking input our program should behave as the desired requirement of the user.

We will be using **Code Widget** for this course where you can always run the program which is already provided to you or you can change and execute it.

[1] Rules that specify what is a valid statement and what is not. Otherwise, we could enter commands in nonsensical ways and the computer will have to reject them.

Example 1: Printing a Welcome Message on the Console Window...!

Let us first write a program that prints a welcome message to display to the user.

Code

```
#include <iostream>
using namespace std;

//This program should print the message inside the quotes
int main()
{
   cout << "Hello Turing… Welcome  :) to the world of C++" << endl;
   return 0;
}
```

Output

```
Hello Turing :) to the world of C++
```

Advice

We strongly advise the following

1. You should first execute the program as it is.
2. Understand the input and output of the program.
3. You should read the description below after every code and understand how the code is executing and understand the computational logic behind every instruction.
4. Execute the program step by step and see the changes in the watches of the editor and how the state of the program is changing with the execution of every instruction in the program.

Description

Here is the line by line details of the code:

```
#include <iostream>
using namespace std;
```

“iostream" is a header file, the above lines are the way to include it in our program. The general syntax is the following.

```
#include <HeaderFileName.h>
```

Or for standard header files

```
#include <HeaderFileName>
using namespace std;
```

Write **using namespace std;** as it is for the time being, we will talk about its details when we talk about **namespaces** in the coming lessons.

Header files are helping library files of your C/C++ program which holds the definitions of various already provided functionalities like taking input from the console and printing on console, and their associated variables that need to be imported into our C/C++ program with the help of pre-processor **#include** statement.

TO REMEMBER

header files are libraries written by other computer scientists and you may include them in your program. Like we included `iostream` which provides the functionality of “cout/cin” to print on the console screen or take input from the user through the console.

```
//This program should print the message inside the quotes
```

Everything followed by // will be considered as comments. This is called a single line comment (as this will convert everything within that line to be considered as comments - and they will not be executed by the operating system.

```
int main()
{
        // Your code must be written here
}
```

Every C++ program must have exactly one **main()**. The lines inside the parentheses **{ }** represent the body of our program which may contain several statements, each line ending with a semicolon. **cout** is a standard syntax to display a text message inside quotations. **cout << "message"** is the syntax for pushing/writing the message on console screen i.e. whatever is written inside the double quotation will be streamed towards console screen as it is; followed by **<<** means another message is cascaded (cascading can be used for printing several messages with one line of instruction) and **endl** is a character for moving to the next line of the console screen.

Note - that every program statement must end with a semicolon. **Also** note that we have used the word push on writing on the console screen; you will figure out why we have said it when we will discuss arrays.

```
cout << "message 1 " << "message 2 " << "message 3" << endl;
```

Which is equivalent to

```
cout << "message 1 ";   // message 1 will be written on console
cout << "message 2 "; // message 2 will be written on console
cout << "message 3 "; // message 3 will be written
cout << endl;          // all the above three messages will be written in one line and the cursor will
                       // move to the line
```

```
return 0;
```

means our program has ended - we will see its further details in the coming lessons. Any statement written after this statement will not be executed.

TO REMEMBER

Don't worry about the syntax errors (in case you miss any semicolon or misspell anything) even if you forget it again and again, that is quite normal and you will get used to it after a while.

Syntax Errors in the Program

```
1  #include <iostream>
2  using namespace std;
3
4  //error in line number
5
6  int main()
7  {
8     cout << "Hello Turing!" << endl
9     return 0;
10 }
```

In the above picture there is a piece of code with an error in line number 8. The red square box indicates that there is an error in line 8. If you will edit the code the line # might be changed so focus on the message where the compiler log is pointing to the error. Usually semicolon ';' missing errors are shown one line after as the compiler realized reading the next instruction that the new statement is starting and just before that a semicolon was expected. These messages vary from compiler to compiler.

Error: Semicolon is missing

```
In function 'int main()':
error: expected ';' before 'return'
=== Build failed: 1 error(s), 0 warning(s) (0 minute(s), 0 second(s)) ===
```

This window from the log window shows errors and warnings. You should correct the error by adding a semicolon in the line just before the return statement and executing again the console must display "Hello Turing" message on the console.

Now let's move to the next examples.

Example 2: Basic Arithmetic Operations(+, *, -)

Let's write a program which can do simple arithmetic operations.

Code

```
1  #include <iostream>
2  using namespace std;
3
4  //This program should do different arithmetic operations
5
6  int main()
7  {
8     cout << 2*3 << endl << 2+3 << " " << 2-3 << endl;
9     return 0;
10 }
```

Output

```
6
5 -1
```

Description

In line 8 **cout** is working in the following fashion. Everything which is written in between two **<<** will be printed one by one. If the message inside **<<** is within double quotes "message" then it will be printed as it is. But if the message is something like a mathematical expression (without a quote) it will be first evaluated (automatically in the background by the executor, usually operating system, during execution) and that evaluated value will be replaced as the message e.g. **2*3** will be replaced by **6** similarly **2+3** will be replaced by **5** and **2-3** will be replaced by **-1**.

Remember

Don't need to use quotations for arithmetic computation, in the above code we use quotations just to give a space between numbers as you see in console output.

Example 3: Table Printing of 23

We want to make a program which should print a table of any number e.g. 23.

We will use what we have learnt in the previous example to write this very simple program. The idea is to use **cout** and print the table line by line. Here is the program.

Code

```
1  #include <iostream>
2  using namespace std;
3
4  //This program should print the table of 23 up to the 10th multiple
5
6  int main()
7  {
8     cout << "23 x  1 = " << 23*1 << endl;    // this line will print          23 x 1 = 23
9     cout << "23 x  2 = " << 23*2 << endl;   // this line will print          23 x 2 = 46
10    cout << "23 x  3 = " << 23*3 << endl;   // execute line by line and check the output
11    cout << "23 x  4 = " << 23*4 << endl;
12    cout << "23 x  5 = " << 23*5 << endl;
13    cout << "23 x  6 = " << 23*6 << endl;
14    cout << "23 x  7 = " << 23*7 << endl;
15    cout << "23 x  8 = " << 23*8 << endl;
16    cout << "23 x  9 = " << 23*9 << endl;
17    cout << "23 x 10 = " << 23*10 <<endl;
18    return 0;
19 }
```

Output

```
23 x 1 = 23
23 x 2 = 46
23 x 3 = 69
23 x 4 = 92
23 x 5 = 115
23 x 6 = 138
23 x 7 = 161
23 x 8 = 184
23 x 9 = 207
23 x 10 = 230
```

Link
http://codepad.org/rXYZZOaf

Description
In this program **cout** is used to print the table of 23 upto 10th multiple. Note that whenever you write something in between the double quotes **"MESSAGE"** that **MESSAGE** will be printed on the console screen as it is. But if we write a mathematical expression without quotes, C++ is intelligent enough that the mathematical expression is replaced by the result of the mathematical expression. For example **23*2** will be replaced by **46** and that result will be printed on the screen.

REMEMBER
Now you have to remember that the material inside the quotes "" is printed on console screen as it is and other computation expression (e.g 23*1, 23*2) is written without quotations for arithmetic computation.

Instruction - Execute the program step by step and look at the console screen after each execution of the instruction.

Exercise 1: Table Printing of 73 upto 10th Multiple

We want to make a program which should print a table of 73 upto 10th multiple.
Modify the previously written program to convert it into printing the table of 73.

Previous code

```
#include <iostream>
using namespace std;

//This program should print the table of 73 up to 10th multiple

int main()
{
   cout << "23 x  1 = " << 23*1 << endl;
   cout << "23 x  2 = " << 23*2 << endl;
   cout << "23 x  3 = " << 23*3 << endl;
   cout << "23 x  4 = " << 23*4 << endl;
   cout << "23 x  5 = " << 23*5 << endl;
   cout << "23 x  6 = " << 23*6 << endl;
   cout << "23 x  7 = " << 23*7 << endl;
   cout << "23 x  8 = " << 23*8 << endl;
   cout << "23 x  9 = " << 23*9 << endl;
   cout << "23 x 10 = " << 23*10<< endl;
   return 0;
}
```

The Expected Output

```
73 x 1 = 73
73 x 2 = 146
73 x 3 = 219
73 x 4 = 292
73 x 5 = 365
73 x 6 = 438
73 x 7 = 511
73 x 8 = 584
73 x 9 = 657
73 x 10 = 730
```

Link
http://codepad.org/Z2C30gCL

Exercise 2

1. Print the table of 35 up-to 20 multiples?

Exercise 3

2. Print the table of 43 from 13 to 30 multiple?

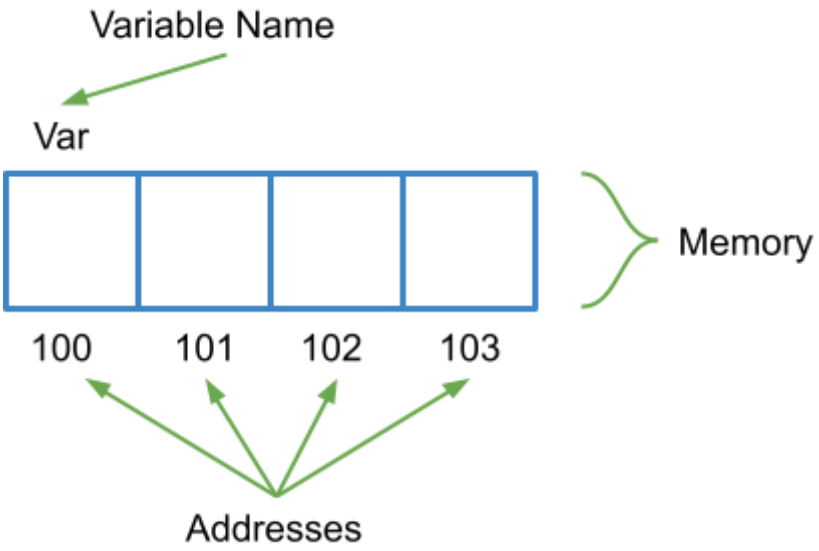
What if we have to print the table of 27 then as you see we changed every 23 or 73 in every line. It is a very cumbersome act to change the code again and again whenever you want to change the table to some other number's table. **Can we do something so that the change in requirement of a different table will require a minimum cumbersome change in the code? This is the fundamental need of writing your software which should allow minimal changes in the software with the change in requirement of the client (the end user).**

Lesson 2 - Variables, Memory and assignment (=) statement

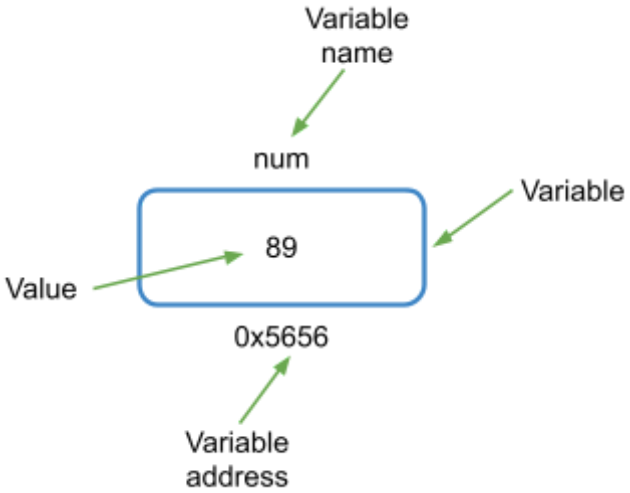
Variables and Memory

Now let's move to a new concept called Variables. Variable is a named location in a computer's memory that stores some value. We can later use that value by the name of the variable. Variables are of different types, which determine the size and layout of a variable's memory, for example, integers, floating-point numbers like real values, character(s), bool (true/1 or false/0), etc. In this chapter, we will only use integer-type variables, so don't worry about the rest. We will learn these in the coming chapters.

For a better understanding of variables, you need to know a little bit about a computer's memory; computer memory is a kind of box aligned up in a row, each box(memory) numbered sequentially. These numbers are known as memory addresses.



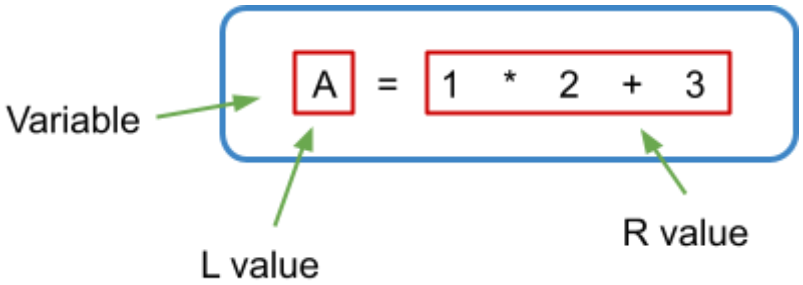
You can also give a name to a memory location that you can later access by its name (compiler in the background manages how to replace the value of the named memory). For example, a variable named **num** in which **89** is stored is shown below.



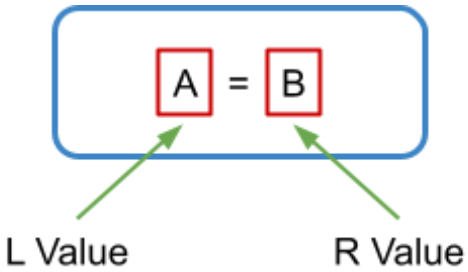
Assignment Statement (=)

The most essential and common statement in C++ is Assignment Statement; in C++ it is used for computation and setting the value to a variable. The symbol that is used for calculation and assigning the value to a variable is called the assignment operator denoted by **=**. This operator (=) is just like the equal sign of mathematics, but in C++ it has a different meaning.

As stated earlier assignment operator is used for computation and assigning the value to a variable. We can set a single value or an expression to a variable as shown in the image below,

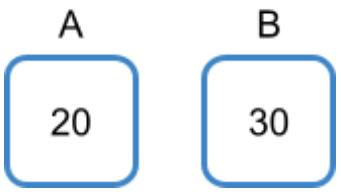


In this image R-value(right value) is assigned to variable A(L value / left value) i.e. 5. You can also assign a value of one variable to another as shown in the image below



Suppose that variable **A** holds **20** and **B** holds **30**.

```
int A = 20;
int B = 30;
```

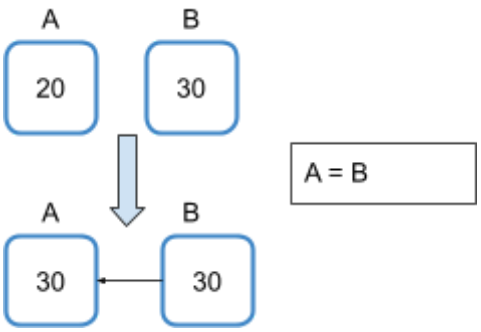


If we have to assign the value of one variable to another we simply write this line of code

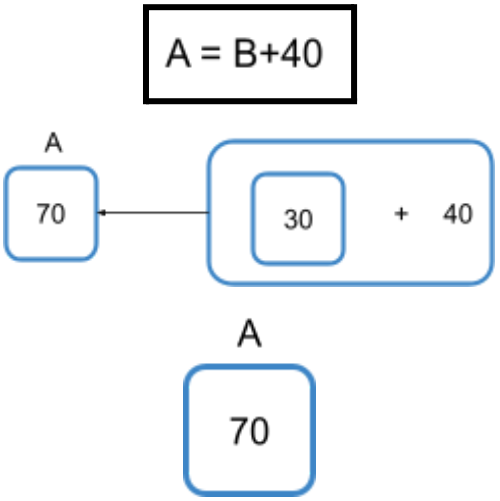
```
A = B;
```

In this line of code **A** is L-value(left value) and **B** is R value(right value). When this line will execute **A** holds exactly the same value as **B** as shown in the image below.

This means **A** will be replaced with a value as of **B**.



Now, for better understanding, we assign an arithmetic expression to a variable.



Variable Declaration and Initialization

To declare any variable, you should first write its data type (i.e., which kind of data the variable will hold) then the variable's name. For example:

int num = 34;

Like in the above example, the variable **num** is initialized with 34. You can also declare multiple variables by using comma separation like below

int num1, num2, num3;

Declaration and initialization process can also be done in one line like this,

int a = 5, b = 3, c = 0;

Variable have different data types, and the computer gives them space(just like a box) according to the data type.

In the picture above, addresses of variables are not shown because the compiler handles it internally, and you don't need to worry about it for now. When we make a variable compiler, assign the address automatically.

The integer is a data type in C++ used to store whole numbers.

Remember
integer type variables do not store fractional numbers (e.g 2.1, 10.6). There is another type used to store such numbers which we will learn in the next chapter.

Rules for naming the variables

In C++ there are some rules for naming a variable:

- Keywords(reserved words) cannot be used as a variable name, e.g., **int, float, double, sizeof, break, for, while,** etc.
    - See the link below for the list of C++ keywords(reserved words) - https://en.cppreference.com/w/cpp/keyword
- Variable names cannot be started with a number or a special character (like $,^, &, @, etc.)
- In C++, variable names must start with alphabets or underscore(_).
- Numbers can be used after the first letter initialization of variables, but space and special characters are still not allowed.

See the link below for the list of special characters.
https://www.owasp.org/index.php/Password_special_characters

Example 5: Integer Variable Declaration, Initialization, and Execution

Let's write a program that declares a variable, initializes and executes it.

Code

```
#include <iostream>
using namespace std;

//This program declare and initialize the integer variable with 34
//num is the name of the variable

int main()
{
   int num;
   num = 34;

   cout << "Number = " << num << endl;
   return 0;
}
```

Output

```
Number = 34
```

Link
http://codepad.org/eYWAeVU2

Description

In this example, an integer type variable is created on line number 9 and initialized with 34 on line number 10. Later on, the number is displayed on the console screen by using **cout** on line number 12.

Example 6: Arithmetic Operations using Integer Variable

In this example, we are going to perform simple arithmetic operations using integer type variables.

Code

```
1  #include <iostream>
2  using namespace std;
3
4  //This program should perform arithmetic operation on variable
5
6  int main()
7  {
8     int num = 34;
9
10    cout << num+1 << endl;
11    cout << num*2 << endl;
12    cout << num-1 << endl;
13    return 0;
14 }
```

Output

```
35
68
33
```

Link

http://codepad.org/C0oWIPhB

Description

As you see in this example from the result, we learned that wherever the variable(**num**) is written, the compiler automatically replaces that expression's spot by its value. The mathematical expression solves, and that statement is replaced with a single value.

Now play with variables, change the value of the variable and see the results.

Program 7.1: Table printing using integer Variable

This is an example of table printing using an integer type variable.

code

```
#include <iostream>
using namespace std;

//This program print the table of 73 up to 10th multiple

int main()
{
   int Table = 73;

   cout << "Table x  1 = " << Table*1 << endl;        // Prints Table x 1 = 73
   cout << "Table x  2 = " << Table*2 << endl;       // Prints Table x 2 = 146
   cout << "Table x  3 = " << Table*3 << endl;
   cout << "Table x  4 = " << Table*4 << endl;
   cout << "Table x  5 = " << Table*5 << endl;
   cout << "Table x  6 = " << Table*6 << endl;
   cout << "Table x  7 = " << Table*7 << endl;
   cout << "Table x  8 = " << Table*8 << endl;
   cout << "Table x  9 = " << Table*9 << endl;
   cout << "Table x 10 = " << Table*10<< endl;
   return 0;
}
```

Output

```
Table x 1 = 73
Table x 2 = 146
Table x 3 = 219
Table x 4 = 292
Table x 5 = 365
Table x 6 = 438
Table x 7 = 511
Table x 8 = 584
Table x 9 = 657
Table x 10 = 730
```

Link

http://codepad.org/eUykwczX

Description

We saw that the **Table** inside the double quotes is not replaced by the value, rather just the label **Table** and the **Table** outside the quotes evaluated and replaced by the value of the variable and then replaced by the answer.

How can we correct this logical error?

Program 7.2: Modified Version of program 7.1

code

```
1  #include <iostream>
2  using namespace std;
3
4  //This program print the table of 73 up to 10th multiple
5
6  int main()
7  {
8     int Table = 73;   // Change the value by 55, 76 and whatever you like to see its effect
9
10    cout << Table << " x  1 = " << Table*1 << endl;
11    cout << Table << " x  2 = " << Table*2 << endl;
12    cout << Table << " x  3 = " << Table*3 << endl;
13    cout << Table << " x  4 = " << Table*4 << endl;
14    cout << Table << " x  5 = " << Table*5 << endl;
15    cout << Table << " x  6 = " << Table*6 << endl;
16    cout << Table << " x  7 = " << Table*7 << endl;
17    cout << Table << " x  8 = " << Table*8 << endl;
18    cout << Table << " x  9 = " << Table*9 << endl;
19    cout << Table << " x 10 = " << Table*10<< endl;
20    return 0;
21 }
22 /*
23 Note: That if now you want to change the Table printing to let us say some other value e.g. 39 then all you
24 need to change is just one line i.e. Line # 8 change it to
25 int Table = #VALUE
26 and you are done with the desired result.
27 */
28
```

Description

We saw in the previous program that the **Table** inside the double quotes does not replace the value, so this time we put the **Table** outside the double quote, and there it changes with its value and replaces it with an answer.

Output

```
73 x  1 = 73
73 x  2 = 146
73 x  3 = 219
73 x  4 = 292
73 x  5 = 365
73 x  6 = 438
73 x  7 = 511
73 x  8 = 584
73 x  9 = 657
73 x 10 = 730
```

Link

http://codepad.org/RRtKV1Qt

Exercise

1. Print the table of 89 with the help of an integer variable up to 25 multiples?
2. Print the table of 43 with the help of an integer variable, from multiple of 11 till 20?

You should have sensed that this is again becoming cumbersome if the user changes the requirement of the table (with a different range). We have to change the program again and again. How can we handle this? That program should prompt the user first which table you would like to print and then print the table according to the input provided?

Taking Input from User (the use of cin)

In C++ **cin** is used to take input from the user and it is predefined in **<iostream>** remember that **cin** is not a keyword. In C++ **cin** and **cout** is standard input and output stream, **cin** is a variable of type istream(input stream), **cin** means standard console input. The user can input the data from the keyboard, file, screen, etc. **cin** is a syntax to take input from the user at run time and store the input value in variables, arrays, and etc. As we see in **cout** insertion operator**(<<)** was used to separate the output message, when using **cin** extraction operator**(>>)** will be used to take input separately.

Program 9: Taking Input from User and Print

This program shows you how to take input from the user and display the result.

Code

```
#include <iostream>
using namespace std;

//This program takes the input from the user and display that number

int main()
{
    int num;
    cout << "Enter number: ";
    cin >> num;

    cout << "Number = " << num << endl;
    return 0;
}
```

Output

```
Enter number: 159
Number = 159
```

Link

http://codepad.org/e22Ss3hA

Description

In the above program **cin** is used to take input from the user and store that input in variable **num** and at the end the same input is displayed on the console screen using **`cout`**.

Remember
If you are taking input for an integer variable then you are bound to write the same type of input.
For example: ***10, 45, 97 etc.***
if you input real numbers like 10.5, 36.9 or 65.7 the integer variable only stores 10, 36 or 65 respectively.

Program 10: Taking Input(number) from User and Print the Table

This is a program of table printing by taking input from the user.

Code

```
#include <iostream>
using namespace std;

//This program should print the table according to user input number

int main()
{
    int Table;
    cout << "Which table you want to Print: ";
    cin >> Table;

    cout << Table << " x 1 = " << Table*1 << endl;
    cout << Table << " x 2 = " << Table*2 << endl;
    cout << Table << " x 3 = " << Table*3 << endl;
    cout << Table << " x 4 = " << Table*4 << endl;
    cout << Table << " x 5 = " << Table*5 << endl;
    cout << Table << " x 6 = " << Table*6 << endl;
    cout << Table << " x 7 = " << Table*7 << endl;
    cout << Table << " x 8 = " << Table*8 << endl;
    cout << Table << " x 9 = " << Table*9 << endl;
    cout << Table << " x 10 = " << Table*10 << endl;
    return 0;
}
```

Output

```
Which table you want to Print: 56
56 x 1 = 56
56 x 2 = 112
56 x 3 = 168
56 x 4 = 224
56 x 5 = 280
56 x 6 = 336
56 x 7 = 392
56 x 8 = 448
56 x 9 = 504
56 x 10 = 560
```

Link
http://codepad.org/SR6BJK6X

Description:

In this program, we use a **Table** variable as integer input, and we can print any table from range 1 to 10. For example, if the user enters 56 as input, it will display a table of 56 from range 1 to 10.

Program 11: Taking Input(number) from User and Print the Table up to 15th Multiple

This program takes input from the user and prints the table of that input(number) upto 15th multiple.

Code

```
#include <iostream>
using namespace std;

//This program should print the table from input number up to 15th multiple

int main()
{
   int Table;
   cout << "Which table you want to Print: ";
   cin >> Table;

   cout << Table << " x 1 = " << Table*1 << endl;
   cout << Table << " x 2 = " << Table*2 << endl;
   cout << Table << " x 3 = " << Table*3 << endl;
   cout << Table << " x 4 = " << Table*4 << endl;
   cout << Table << " x 5 = " << Table*5 << endl;
   cout << Table << " x 6 = " << Table*6 << endl;
   cout << Table << " x 7 = " << Table*7 << endl;
   cout << Table << " x 8 = " << Table*8 << endl;
   cout << Table << " x 9 = " << Table*9 << endl;
   cout << Table << " x 10 = " << Table*10 << endl;
   cout << Table << " x 11 = " << Table*11 << endl;
   cout << Table << " x 12 = " << Table*12 << endl;
   cout << Table << " x 13 = " << Table*13 << endl;
   cout << Table << " x 14 = " << Table*14 << endl;
   cout << Table << " x 15 = " << Table*15 << endl;
   return 0;
}
```

Output

```
Which table you want to Print: 56
56 x 1 = 56
56 x 2 = 112
56 x 3 = 168
56 x 4 = 224
56 x 5 = 280
56 x 6 = 336
56 x 7 = 392
56 x 8 = 448
56 x 9 = 504
56 x 10 = 560
56 x 11 = 616
56 x 12 = 672
56 x 13 = 728
56 x 14 = 784
56 x 15 = 840
```

Link
http://codepad.org/0pmZrgLu

Description
In the above code, we have a print table from 1 to 15. If we would like to print the table up to 50th multiple or 100'th multiple, then we have to write 50 or 100 times the lines with multiplicand changes in each line.

To avoid this repetition of a similar code, we will learn an extremely powerful construct of repetition called a loop.

Repetition and **while** loop

To repeat a process (set of statements) in C++ we use loops. Loops make our work a lot easier and make our code smarter. There are different kinds of loops, **while** is one of its types.

It's syntax is,

```
while(condition)
{
        Body(statements) of while
}
```

If the **condition** is true, then the Body part of the loop will be executed. After completely executing the Body part, the **condition** of the **while** loop will be checked again. If it is true, then the body will be executed again, and this process will continue until the condition becomes false.

In this program below the message **Hello** print continuously.

Program 12: Infinite Hello Printing using While loop

This program prints the message continuously(infinite time) using a while loop.
Code

```
#include <iostream>
using namespace std;

//This program should print the quoted message infinitely

int main()
{
   while(true)
   {
      cout << "Hello" << endl;
   }
   return 0;
}
```

Output

```
Hello
Hello
Hello
Hello
Hello
Hello
Hello
Hello
Hello
```

Link: http://codepad.org/xKtC2kvx

Description

You can terminate the infinite loop by pressing the **ctrl + c** keys or by closing the console window.

Program 13: Printing Hello 10 Times using While loop

This program print the value inside the quotes and also show the incremented value of time.

Code

```
#include <iostream>
using namespace std;
/*This program should print the quoted message until the condition of while loop get false.
In this program while loop runs 10 times with integer variable*/
int main( )
{
   int Time = 1;
   while(Time  <= 10)
   {
      cout << Time << ": hello" << endl;
      Time = Time + 1;
   }
   return 0;
}
```

Output

```
1: hello
2: hello
3: hello
4: hello
5: hello
6: hello
7: hello
8: hello
9: hello
10: hello
```

Link

http://codepad.org/JgV6SWic

Line by line Description

1.	`int Time = 1,`	**Time** variable initialised with 1
2.	`while(Time  <= 10)`	The loop will be executed until Time is not equal and less than 10
3.	`{`	If the condition is true everything inside the block {...} will be executed
4.	`cout << Time << ": hello" << endl;`	It will displays hello
5.	`Time = Time + 1;`	It will increment in time variable by 1 on each iteration
6.	`}`	Again move to step 2 until condition false
7.	`return 0;`	

Most of the time if you exactly know how many times you want to repeat this process the **while** loop is not preferred. The preferred loop is the **for** loop. Let us look at it.

Remember

If the loop repeats its iteration continuously and never stops this type of loop is called an infinite loop. An infinite loop must be avoided

FOR loop

Its syntax is,

```
for(initialization; condition; increment or decrement)
{
        Body(statements) of for loop
}
```

For Loop runs in the following way:

- Step 1. It will declare and initialize the control variable.
- Step 2. The condition expression is evaluated and two cases may arise,
  - True - resulting in control to move to Body part
  - False - resulting in the for loop to break and control going after the loop.
- Step 3. Body part is executed only if the condition is true and after finishing the control goes back to the loop with increment or decrement part of the loop.
- Step 4. After executing a statement, it will increase or decrease the control variable.
- Step 5. Now repeat Steps 2 – 4

Program 13.1: Printing Hello 10 Times using For loop

This program prints the value inside the quotes and also shows the incremented value of time.

Code

```
#include <iostream>
using namespace std;

/*This program should print the quoted message until the condition of for loop get false.
In this program for loop runs 10 times with integer variable*/

int main()
{
   for(int Time = 1; Time <= 10; Time++)
   {
      cout << Time << ": hello" << endl;        // On every iteration Time will be replaced by its value
   }
   return 0;
}
```

Output

```
1: hello
2: hello
3: hello
4: hello
5: hello
6: hello
7: hello
8: hello
9: hello
10: hello
```

Link: http://codepad.org/83PwQgCG

Description

This program will print a message hello 10 times on 10 lines.

Program 14: Table Printing using While Loop

This program prints the table of user input number upto 15th multiple using a while loop.

code

```
#include <iostream>
using namespace std;

//This program should print the table up to 15th multiple
//Table printing using while loop according to user input

int main()
{
    int Table;
    cout << "Which Table: ";
    cin >> Table;          //Let us say user entered 12

    int i=1;
    while(i <= 15)
    {
        cout << Table << " x " << i << " = " << Table*i << endl;
        i = i + 1;
    }
    return 0;
}
```

Output

```
Which Table: 12
12 x 1 = 12
12 x 2 = 24
12 x 3 = 36
12 x 4 = 48
12 x 5 = 60
12 x 6 = 72
12 x 7 = 84
12 x 8 = 96
12 x 9 = 108
12 x 10 = 120
12 x 11 = 132
12 x 12 = 144
12 x 13 = 156
12 x 14 = 168
12 x 15 = 180
```

Link

http://codepad.org/6PayfOwd

Description

We want to print the table by the user's choice(input) up to 15th multiple in program. In line 16 we write a specific format to print table, and then we multiply table num with the number of iteration. On the next line, we just increment in the iteration variable **i** so that every time the table multiplies with an incremented value of **i**.

Program 14.1: Table Printing using For Loop

This program print the table of user input number upto 15th multiple using for loop.

Code

```
1  #include <iostream>
2  using namespace std;
3  //This program should print the table up to 15th multiple
4  //Table printing using while loop according to user input
5
6  int main()
7  {
8     int Table;
9     cout << "Which Table: ";
10    cin >> Table;        //Let us say user entered 12
11
12    for(int i =  1; i <= 15; i++)
13    {
14       cout << Table << " x " << i << " = " << Table*i << endl;
15    }
16    return 0;
17 }
```

Output

```
Which Table: 12
12 x 1 = 12
12 x 2 = 24
12 x 3 = 36
12 x 4 = 48
12 x 5 = 60
12 x 6 = 72
12 x 7 = 84
12 x 8 = 96
12 x 9 = 108
12 x 10 = 120
12 x 11 = 132
12 x 12 = 144
12 x 13 = 156
12 x 14 = 168
12 x 15 = 180
```

Link: http://codepad.org/wyluAOqK

Description

We want to print the table by the user's choice(input) up to 15 in program. In line 14 we write a specific format to print table, and then we multiply table num with the number of iteration. Each iteration loop will increment **i** by 1 until condition false **i <= 15**.

Program 15: Table Printing up to User Entered Limit using While Loop

This program takes two input from user about the table and limit(multiples) and print the table of user entered number upto user entered limit(multiples).

code

```
1  #include <iostream>
2  using namespace std;
3
4  //Table printing up to the limit(multiples) entered by user
5  //While loop that print table according to user input
6
7  int main()
8  {
9     int Table;
10    int Limit;
11
12    cout << "Which Table: ";
13    cin >> Table;          //Let us say user entered 5
14    cout << "Limit: ";
15    cin >> Limit;          //let us say user entered 16
16
17    int i=1;
18    while(i <= Limit)
19    {
20       cout << Table << " x " << i << " = " << Table*i << endl;
21       i = i + 1;
22    }
23    return 0;
24 }
```

Output

```
Which Table: 5
Limit: 16
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
5 x 11 = 55
5 x 12 = 60
5 x 13 = 65
5 x 14 = 70
5 x 15 = 75
5 x 16 = 80
```

Link

http://codepad.org/lssSef3n

description

In this program, three variables are used. One is **Table** that is used for which Table you want to print, the Second is **i** as a counter, and the last one is as a **Limit** that is exactly how many times the loop will be executed.

- In lines 13 and 15 we are getting value for **Table** and **Limit**.
- In line 17 the counter variable **i** is assigned with 1.
- In line 18 the **while** condition will execute until **i** is less than equal to **Limit**.
- In line 20 it will print the table on the console on the basis of **Table** value and **Limit.**

Program 15.1: Table Printing up to User Entered Limit using For Loop

This program takes two inputs from user about the table and limit(multiples) and print the table of user entered number upto user entered limit(multiples).

Code

```
1  #include <iostream>
2  using namespace std;
3  //Table printing up to the limit(multiples) entered by user
4  //For loop that print table according to user input
5
6  int main()
7  {
8     int Table;
9     int Limit;
10    cout << "Which Table: ",
11    cin >> Table;          //Let us say user entered 5
12    cout << "Limit: ";
13    cin >> Limit;          //let us say user entered 16
14
15    for(int i = 1; i <= Limit; i++)
16    {
17       cout << Table << " x " << i << " = " << Table*i << endl;
18    }
19    return 0;
20 }
```

Output

```
Which Table: 5
Limit: 16
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
5 x 11 = 55
5 x 12 = 60
5 x 13 = 65
5 x 14 = 70
5 x 15 = 75
5 x 16 = 80
```

Link: http://codepad.org/qup4EWKf

Description

In this program, we will do the same task using For Loop, So three variables are used. One is **Table** that is used for which Table you want to print, the Second is **i** as a counter, and the last is as a **Limit** that is exactly how many times the loop will be executed.

- In line 11 and 13 we are getting value for **Table** and **Limit**.
- In line 15, In for loop, we are initializing value **i** with 1, the second argument we are checking condition **i** less than equal to **Limit,** and last argument, we are incrementing **i** by 1.
- In line 17 it will print the Table on the console on the basis of **Table** value and **Limit**.

Program 16: Generic Table Printing (User Entered Table and Limits from start to end)

This program is slightly different from previous one this program takes three inputs from the user about table, starting multiple and ending multiple. In short it is generic program.

Code

```
1  #include <iostream>
2  using namespace std;
3
4  //Table printing within the user entered limit(multiple)
5  //example of while loop that print table according to user input
6
7  int main()
8  {
9     int Table;
10    int Start, End;
11
12    cout << "Which Table: ";
13    cin >> Table;            //Let say user entered 7
14    cout << "Start: ";
15    cin >> Start;            //Let say user entered 8
16    cout << "End: ";
17    cin >> End;              //Let say user entered 15
18                        // printing should begin from Start
19    int i=Start;
20
21    while(i <= End)
22    {
23       cout << Table << " x " << i << " = " << Table*i << endl;
24       i = i + 1;
25    }
26    return 0;
27 }
```

Output

```
Which Table: 7
Start: 8
End: 15
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
7 x 11 = 77
7 x 12 = 84
7 x 13 = 91
7 x 14 = 98
7 x 15 = 105
```

Link

http://codepad.org/y0MJVVPb

Description

This program is slightly different from the previous While loop example. We are taking **Table, Start** and **End** from the user. The **table** variable is used to print which table we want to print; the **Start** variable is assigned to **i**, which represents where we start the table, and **End** represents the limit from where we end the table.

Remember

Generic programming means that you can easily reuse that program without changing the code. In short user friendly program.

Program 16.1: Generic Table Printing (Using For loop)

Let us do the same program using **for** loop instead of **while**.

Code

```
#include <iostream>
using namespace std;
//Table printing within the user entered limit(multiple)
/*example of for loop that print table according to
user input within given limits*/

int main()
{
    int Table;
    int Start, End;
    cout << "Which Table: ";
    cin >> Table;            //Let say user entered 7
    cout << "Start: ";
    cin >> Start;            //Let say user entered 8
    cout << "End: ";
    cin >> End;              //Let say user entered 15

    // printing should begin from Start till End
    for(int i=Start; i<=End; i++)
    {
        cout << Table << " x " << i << " = " << Table*i << endl;
    }
    return 0;
}
```

Output

```
Which Table: 7
Start: 8
End: 15
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
7 x 11 = 77
7 x 12 = 84
7 x 13 = 91
7 x 14 = 98
7 x 15 = 105
```

Link: http://codepad.org/vXIJ8y65

Description

It is the exact same Program we done in previous example, but we done this using for loop.